

9 调度问题的语言：一句话说清是什么问题，有多难

前两部分回答“谁做、怎么走”。这一部分回答“什么时候做”：提升机、站点、堆垛设备都是要排队的机器。调度领域有一套简洁的记号，一个三段式的标签就能说清机器怎么配置、有什么约束、要优化什么。学会读写这个标签，就能在文献里迅速找到你的问题，知道它是容易的还是难的。

学完本章，你能

- 1. 用作业、机器、调度、甘特图描述一个排班问题，区分“分配、排序、定时”三类决策。
- 2. 读写 $\alpha | \beta | \gamma$ 三段式记号，把提升机、多车、站点串联这些场景翻译成标签。
- 3. 计算 $C_{\max}$ 、 $\sum C_j$ 、 $\sum w_j C_j$ 、 $L_{\max}$ 、 $\sum T_j$ 、 $\sum U_j$ ，并解释为什么没有一种排法能让它们同时最好。
- 4. 用“特例与推广”的关系推理一个问题的复杂度，读复杂度地图，知道该去用规则、动态规划、还是启发式与求解器。

本章信息

用时	阅读与交互 70-100 分钟，Lab 2.5 小时左右
前置	第 1 章（建模）、第 5、6 章（指派、网络流）、第 8 章（NP 难的直观）。会读 Python 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0；本章的算法由页面内置代码实现，不依赖求解器。
配套	Lab 代码包：5 个练习 + 1 个挑战，含目标函数评估、并行机列表调度、记号解析与复杂度推理，pytest 自动判分

PART 1 数学规划基础	PART 2 组合优化	PART 3 调度理论	PART 4 路径与波次
PART 5 大规模方法	PART 6 启发式	PART 7 多智能体时空规划	PART 8 动态与不确定

第三部分 调度理论：调度问题的语言（本章）→ 单机与并行机 → 流水车间与作业车间 → 资源与时间窗约束。

9.1 什么是调度问题

6 个托盘要过同一台提升机。每个托盘占用提升机若干秒，有的着急、有的不急，有的必须赶上某个发运截止时间。谁先上、谁后上？这就是一个调度问题。

调度问题的基本语言只有三样：

作业 (job)

要完成的活，编号 j 。每个作业有：加工时间 p_j （占用机器多久）；释放时间 r_j （最早什么时候能开始）；交货期 d_j （最好什么时候完成）；权重 w_j （有多重要）。

机器 (machine)

做活的资源。提升机、堆垛机、装卸月台、四向车，在调度里都可以当作机器：同一时刻只能做一个作业。

调度 (schedule)

给每个作业指定在哪台机器上、什么时候开始。画出来是**甘特图**：横轴是时间，每台机器一行，每个作业是一个方块。作业 j 的完工时间记作 C_j 。

做一个调度，其实是做三类决策：**分配**（每个作业去哪台机器）、**排序**（同一台机器上先做谁）、**定时**（每个作业几点开始）。单台机器、没有释放时间的时候，只剩排序：6 个作业有 $6! = 720$ 种顺序。

本章反复用到的例子：一台提升机，6 个托盘。

作业	J1	J2	J3	J4	J5	J6
加工时间 p	7	3	5	2	8	5
权重 w	2	2	4	4	4	2
交货期 d	13	23	17	5	9	21

总加工时间 30，所以不管怎么排，最后一个作业都在时刻 30 结束。但每个作业各在什么时候完成，取决于顺序，好坏也就取决于你在乎什么，这就是下一节要回答的问题。

9.2 三段式记号

Graham 等人（1979）给调度问题定了一套记号： $\alpha \mid \beta \mid \gamma$ 。三段分别说机器、约束、目标。一句话能说清楚一个问题，也是查文献的关键词。

段	含义	常见取值
α 机器环境	有几台机器，怎么配置	1 单机； Pm 相同并行机； Qm 速度不同的并行机； Rm 不相关并行机； Fm 流水车间； Jm 作业车间； Om 开放车间
β 约束与特征	作业的附加条件，可以为空或多个	r_j 有释放时间； prec 先后约束； prmp 允许中断； d_j 交货期是硬性的； s_{jk} 准备时间（与顺序有关）
γ 目标	要最小化什么	C_{max} 、 ΣC_j 、 $\Sigma w_j C_j$ 、 L_{max} 、 ΣT_j 、 ΣU_j

几个例子，读一遍，感受一下：

1 | | ΣC_j ：一台机器，没有额外约束，让所有作业的完工时间之和最小（平均等待最短）。

1 | r_j | L_{max} ：一台机器，作业陆续到达，让最晚的那个比交货期迟得最少。

P2 | | C_{max} ：两台相同的机器，让最后一个作业尽快完成（均衡负荷）。

R | | ΣC_j ：不相关并行机，每个作业在不同机器上时间不同，让完工时间之和最小。

F2 | | C_{max} ：两台机器串联的流水车间，每个作业先过第一台再过第二台。

后面 9.5 节的图 9-3 让你自己拼一个标签，它会告诉你这个问题在文献里是容易还是困难。

9.3 目标函数：你在乎什么

“排得好”没有唯一的意思。同一批作业，追求的东西不同，最优的顺序也不同。常见的目标都由完工时间 C_j 和交货期 d_j 算出来：

目标	公式	在乎什么
最大完工时间 (makespan)	$C_{max} = \max C_j$	所有活多久干完，设备利用率
完工时间之和	$\sum C_j$ 或 $\sum w_j C_j$	平均等待，在制品库存；权重表示重要程度
最大延迟 (lateness)	$L_{max} = \max(C_j - d_j)$	最坏的那一单迟了多少；可以是负数（提前）
延误时间之和 (tardiness)	$\sum T_j, T_j = \max(0, C_j - d_j)$	迟到总共多久，提前不奖励
延误作业个数	$\sum U_j, U_j = 1$ 若 $C_j > d_j$	有几单没赶上截止时间

这些目标都是**正则**的：完工时间变大，目标不会变好。所以没有必要故意让机器空转，最优调度里作业总是尽早开始。

先预测 6 个托盘在同一台提升机上排队，按加工时间从短到长排 (**SPT**)。不计 **C_max** (它与顺序无关)，**SPT** 排出来的顺序，在 **$\sum C$** 、 **$\sum wC$** 、**L_max**、 **$\sum T$** 、 **$\sum U$** 这五个目标里，有几个是最优的？

0 个

1 个 (**$\sum C$**)

3 个

全部 5 个

预测正确。 只有 $\sum C_j : 84$ ，是最优。 $\sum wC_j$ 要看权重 (**WSPT** 才最优)，**L_max** 要看交货期 (**EDD**)， $\sum U_j$ 要用 Moore-Hodgson。没有哪一种顺序能同时最好。下面的图已经解锁。

套用规则

先来先做

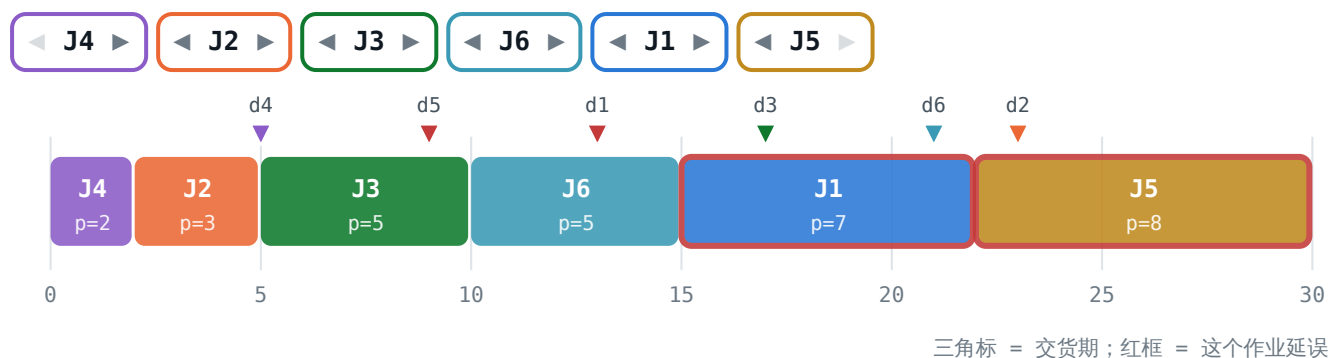
SPT 最短优先

WSPT 加权最短

EDD 交货期

Moore-Hodgson

当前顺序（点两侧的箭头调整）：



目标

当前

最优

取得最优的顺序

最大完工时间 C_{max}	30	30	720 个顺序，如 J1 J2 J3 J4 J5 J6
完工时间之和 $\sum C_j$	84	84	2 个顺序，如 J4 J2 J3 J6 J1 J5
加权完工时间之和 $\sum w_j C_j$	252	234	J4 J3 J2 J5 J6 J1
最大延迟 L_{max}	21	7	2 个顺序，如 J4 J5 J1 J3 J6 J2
延误时间之和 $\sum T_j$	30	18	J4 J5 J3 J6 J2 J1
延误作业个数 $\sum U_j$	2	1	J4 J1 J3 J6 J2 J5
加权延误作业数 $\sum w_j U_j$	6	4	J4 J1 J3 J6 J2 J5

当前顺序：**J4 → J2 → J3 → J6 → J1 → J5**。它在 **$\sum C_j$** 上是最优的（不计 C_{max} ，因为没有释放时间时 C_{max} 与顺序无关）。

没有一种顺序能让所有目标同时最好：SPT、WSPT、EDD、Moore 各自是一类目标的最优规则，换一个目标就换一种“最好”的顺序。选目标，是在选调度的方向。

图 9-1 同一台提升机上的 6 个作业。方块是作业（宽度是加工时间），三角标是交货期，红框表示这个作业延误。可以套用规则，也可以自己调整顺序。表里“最优”一列是枚举全部 720 种顺序得到的，绿色的行表示当前顺序在这个目标上已经最优。

读图

SPT（最短加工时间优先）：J4、J2、J3、J6、J1、J5。它让 $\sum C_j$ 达到 84，这是最优（比先来先做的 104 少了近两成），但 L_{max} 是 21，有 2 个作业延误。**WSPT（加权最短优先）** 让 $\sum w_j C_j$ 最优（234），此时重要的作业排在前面。**EDD（最早交货期优先）** 让 L_{max} 最小（7），但 $\sum C_j$ 涨到

108，延误作业的个数是 5。**Moore-Hodgson** 只有一个作业延误 ($\sum U_j = 1$ ，最优)，代价是 $L_{\max}$ 高达 21：它干脆把那个赶不上的作业放到最后。

所以目标函数决定了你该用什么规则。这些规则 (SPT、WSPT、EDD、Moore) 都是精确的多项式算法，下一章会逐个证明它们为什么对。

把 $\sum C_j$ 和 $L_{\max}$ 放在一起看：它们冲突。所有顺序里，两个目标的帕累托前沿有 8 个点，从 ($\sum C = 108, L_{\max} = 7$) 到 (84, 21)，想让最坏的一单迟得少，就得让平均等待变长。到底选哪一个点，是业务的决定，不是算法的决定。

9.4 机器环境：有几台，怎么配

机器不止一台的时候，先要分清机器之间是什么关系。

并行机：一个作业只在一台机器上做一次

P (相同)：加工时间与机器无关。**Q** (速度不同)：机器 i 的速度是 s_i ，时间是 p_j / s_i 。**R** (不相关)： p_{ij} 是任意的一张表，机器各有所长，比如不同位置的四向车做同一个任务的行驶时间。

车间：一个作业要依次经过几台机器

F 流水车间：所有作业经过机器的顺序相同 (入库 → 提升机 → 出库)。**J** 作业车间：每个作业有自己的路线。**O** 开放车间：作业要在每台机器上做，顺序不限。第 11 章展开。

P、Q、R 越往后越一般：P 是 Q 的特例 (所有速度为 1)，Q 是 R 的特例 ($p_{ij} = p_j / s_i$)。这个“特例关系”后面用来推理复杂度。

先预测 6 个作业的加工时间是 7、3、5、2、8、5，总量 30，放到两台相同的机器上，下界是 15。按编号顺序，逐个放到完工最早的机器上 (列表调度)，**C_max** 是多少？

15，恰好达到下界

16

18

20

预测正确。 16，比最优 15 多 1。列表调度每一步只看“放哪里现在完工最早”，最后一个作业 J6 落在已经有 11 的机器上。改成最长的先放 (LPT) 就得到 15。下面的图已经解锁。



图 9-2 同一批 6 个作业（加工时间 7、3、5、2、8、5）在不同机器环境下，用“列表调度”排：按给定顺序，逐个把作业放到完工最早的机器上。橙色虚线是 $C_{\max}$ 。最优值由枚举每个作业去哪台机器得到。

读图

单机： $C_{\max} = 30$ ，总加工时间，顺序无关。**两台相同的机器：**总量 30，下界 15。按编号列表调度得到 16，最优是 15；改成“最长的先放”（LPT）就得到 15。列表调度是一种很快的启发式，但它没有保证一定最优，即使是最简单的 $P2 \parallel C_{\max}$ ，它也是一个 NP 难的问题（由划分问题推出）。

两台不相关的机器：按编号列表调度恰好是最优（11）；换成“最长的先放”，得到 14，比最优多 3。**顺序策略对不同环境的效果不同**，没有哪一条规则永远最好，所以要知道问题是哪一类，才能知道该用哪种方法。

9.5 复杂度地图：哪些容易，哪些难

调度领域积累了几百个问题的复杂度结果。你不需要背，只需要理解推理规则：一个问题至少和它的特例一样难，至多和它的推广一样容易。

- 1 **特例与推广**：1 是 P 的特例，P 是 Q 的特例，Q 是 R 的特例；没有释放时间是有释放时间的特例； $C_{\max}$ 是 $L_{\max}$ 的特例（令 $d_j = 0$ ）； ΣC_j 是 $\Sigma w_j C_j$ 的特例（令所有 $w_j = 1$ ）。
- 2 **难度沿推广传播**：如果 A 是 NP 难的，B 是 A 的推广，B 也是 NP 难的。因为解 B 的算法可以拿来解 A。
- 3 **容易沿特例传播**：如果 B 有多项式算法，A 是 B 的特例，A 也有。
- 4 **但可中断 (prmp) 不在这个序里**：允许中断可能让问题变容易，也可能没有影响，不能靠它推理。

“NP 难”也有强弱：**弱 NP 难**的问题有伪多项式时间的算法（数值不大时可以用，比如动态规划），**强 NP 难**的问题连这样的算法也没有（除非 $P = NP$ ）。图 9-3 里橙色是弱 NP 难，红色是强 NP 难或没有区分。

先预测 $R \parallel \Sigma C_j$ ：不相关并行机，每个作业在不同机器上的加工时间各不相同，最小化完工时间之和。它属于哪一类？

多项式可解

弱 NP 难

强 NP 难

复杂度未知

预测正确。 多项式可解：把（作业 j 排在机器 i 倒数第 k 位）当作“指派”，位置越靠后对总和的贡献越大，就是一个指派问题（第 5 章）。而同一环境下的 $R \parallel C_{\max}$ 是强 NP 难。同样的机器，换一个目标，难度完全不同。下面的图已经解锁。

α 机器

1

P2

Pm

Qm

Rm

F2

Fm

J2

Jm

O2

Om

β 约束

rj

prec

prmp

dj

sjk

 γ 目标

Cmax

ΣC_j

$\Sigma w_j C_j$

Lmax

ΣT_j

$\Sigma w_j T_j$

ΣU_j

$\Sigma w_j U_j$

你拼出的问题 1 || ΣC_j

多项式可解

表中的已知结果：SPT（最短加工时间优先）。

	$C_{\max}$	ΣC_j	$\Sigma w_j C_j$	$L_{\max}$	ΣT_j	$\Sigma w_j T_j$	ΣU_j	$\Sigma w_j U_j$
1	P	P	P	P	弱	难	P	弱
P2	弱	P	?	弱	弱	难	?	弱
Pm	难	P	难	难	弱	难	?	弱
Qm	难	P	难	难	弱	难	?	弱
Rm	难	P	难	难	弱	难	?	弱
F2	P	难	难	?	难	难	?	弱
Fm	难	难	难	难	难	难	?	弱
J2	P	难	难	?	难	难	?	弱
Jm	难	难	难	难	难	难	?	弱
O2	P	?	?	?	弱	难	?	弱
Om	难	?	?	难	弱	难	?	弱

β 为空：没有释放时间、先后约束等附加条件。

多项式可解 P

弱 NP 难（伪多项式）

NP 难（强）

表里推不出

图 9-3 三段式记号与复杂度地图。上面选 α （机器）、 β （约束）、 γ （目标），左边给出这个问题的分类和理由；右边的地图是：在当前选的 β 下，每种（机器 $\times$ 目标）的组合是容易还是难，点格子可以切换。分类来自一张经典结果表（30 多个条目），加上上面的“特例与推广”推理；鼠标停在格子上能看到理由。

读图

先看 β 为空的地图：单机（第一行）上，目标几乎全是多项式可解的： $C_{\max}$ 、 ΣC_j 、 $\Sigma w_j C_j$ 、 $L_{\max}$ 、 ΣU_j 都有简单的规则。往下走，机器变多，绿色越来越少：**P2 的 $C_{\max}$ 已经是 NP 难**（划

分问题)。同一个机器环境里，目标不同，难度也可以完全不同： $R \parallel \sum C_j$ 是多项式的（它就是第 5 章的指派问题）， $R \parallel C_{\max}$ 却是强 NP 难。

再加 β ：点亮 r_j （释放时间）。单机的 $L_{\max}$ 、 $\sum C_j$ 立刻变成红色：没有释放时间时 EDD、SPT 一步到位，一旦作业陆续到达，问题就变成强 NP 难。这个变化对系统很重要：**任务动态到达，会让原本简单的排序问题变难**，也是第 30 章在线调度的出发点。

复杂度地图的用法

遇到一个调度问题，先写出它的三段式标签，查地图。绿色：找对应的规则或算法，不要建 MIP。橙色：数值不大时用动态规划。红色：用 MIP 或约束规划（第 11 章）求解小规模，用启发式（第 21 章）求解大规模，并用下界评价解的质量。

9.6 在项目里的位置

把你场景里的设备翻译成三段式标签，是这一部分每一章开始前要做的事。

场景	记号	难度与做法
一台提升机，托盘排队，尽量减少平均等待	$1 \mid \mid \Sigma C_j$ (或带权重 $\Sigma w_j C_j$)	多项式：SPT / WSPT (第 10 章)
一台提升机，托盘要赶发运截止时间	$1 \mid \mid L_{\max}; 1 \mid \mid \Sigma U_j$	多项式：EDD；Moore-Hodgson (第 10 章)
托盘陆续到达提升机（不知道后面会来什么）	$1 \mid r_j \mid L_{\max}$	强 NP 难；在线时更难 (第 30 章)
两台提升机分担同一批托盘	$P2 \mid \mid C_{\max}$	NP 难 (弱)；LPT 有 7/6 的近似 (第 10 章)
多辆四向车，各自到每个任务的时间不同	$R \mid \mid \Sigma C_j$	多项式：指派问题 (第 5 章)
入库 → 提升机 → 出库，三个工位串联	$F3 \mid \mid C_{\max}$	强 NP 难；MIP 或启发式 (第 11 章)
设备在两个作业之间要移动，移动时间取决于前后两个作业	$1 \mid s_{jk} \mid C_{\max}$	就是旅行商问题 (第 8 章)
提升机、站点是共享资源，有时间窗	超出经典记号，需要资源约束的扩展	RCPSP、时间索引 MIP (第 12 章)

其中最后一项提醒：三段式记号覆盖的是经典模型。真实系统里的约束（时间窗、共享资源、设备之间的互斥）往往超出记号，这时它的价值是让你说清“最接近的经典问题是什么”，然后知道自己在它的基础上加了什么、复杂度大致会怎样变化。

9.7 小结与 Lab

- 1 调度问题由作业 (p 、 r 、 d 、 w)、机器和调度（分配、排序、定时）组成，用甘特图表示。提升机、堆垛机、月台、四向车在调度里都是“机器”。
- 2 三段式记号 $\alpha \mid \beta \mid \gamma$ 分别是机器环境、约束、目标： 1 、 P 、 Q 、 R 是并行机， F 、 J 、 O 是车间； β 里常见 r_j 、 prec 、 prmp 、 s_{jk} ； γ 是要最小化的目标。一个标签能说清一个问题，也是查文献的关键词。
- 3 目标不同，最优的排法不同： ΣC_j 偏爱 SPT， $\Sigma w_j C_j$ 偏爱 WSPT， $L_{\max}$ 偏爱 EDD， ΣU_j 偏爱 Moore-Hodgson。没有一种排法能让所有目标同时最好； ΣC_j 与 $L_{\max}$ 的帕累托前沿有 8 个

点。选目标是业务的决定。

- 4 $C_{\max}$ 在没有释放时间的单机上与顺序无关（总加工时间）；并行机上它是均衡负荷问题，列表调度（LPT 等）是快速的启发式，但不保证最优。
- 5 复杂度靠“特例与推广”推理：问题至少和它的特例一样难，至多和它的推广一样容易；可中断（prmp）不在这个序里。单机上大部分目标有简单的规则，机器一多、加上释放时间，许多问题就变成 NP 难；同一个机器环境里目标不同，难度可以相差悬殊（ $R \parallel \sum C_j$ 是指派， $R \parallel C_{\max}$ 强 NP 难）。
- 6 先写标签、查地图，再决定用规则、动态规划、MIP 还是启发式。经典记号覆盖不了的约束（时间窗、共享资源），也要先找“最接近的经典问题”。

术语表

中文	English	在本章的含义
作业、机器	job, machine	要做的活；同一时刻只能做一个作业的资源
加工时间、释放时间、交货期、权重	p_j, r_j, d_j, w_j	占用机器多久；最早开始；希望完成的时刻；重要程度
完工时间	completion time C_j	作业 j 做完的时刻
甘特图	Gantt chart	横轴是时间，每台机器一行的排班图
三段式记号	$\alpha \mid \beta \mid \gamma$ notation	机器环境 约束 目标
最大完工时间	makespan $C_{\max}$	最后一个作业的完工时间
延迟、延误	lateness L_j , tardiness T_j	$C_j - d_j$ (可为负) ; $\max(0, C_j - d_j)$
正则目标	regular objective	完工时间变大目标不会变好
并行机 P/Q/R	identical / uniform / unrelated machines	相同；速度不同；加工时间是任意的表 p_{ij}
流水车间、作业车间、开放车间	flow shop, job shop, open shop	路线相同；路线各异；路线不限
列表调度	list scheduling	按给定顺序，把作业放到完工最早的机器
弱/强 NP 难	NP-hard in the ordinary / strong sense	有 / 没有伪多项式算法 (除非 $P = NP$)
帕累托前沿	Pareto front	没有被别的排法在两个目标上同时压过的排法

对照教材

- Pinedo , *Scheduling: Theory, Algorithms, and Systems* (第 5 版 , Springer 2016) : 第 1-2 章调度问题的分类与记号，附录的复杂度表。本课程调度部分的主要参考。
- Graham、Lawler、Lenstra、Rinnooy Kan , “Optimization and approximation in deterministic sequencing and scheduling: a survey” (Annals of Discrete Mathematics , 1979) : 三段式记号的出处。

- Brucker , *Scheduling Algorithms* (第 5 版 , Springer 2007) : 各类问题的算法与复杂度分类 , 含完整的复杂度图。
- Lawler、Lenstra、Rinnooy Kan、Shmoys , “Sequencing and scheduling: algorithms and complexity” (Handbooks in OR & MS , 1993) ; Garey & Johnson , *Computers and Intractability* (1979) : NP 难的基础。
- 网上的“Scheduling Zoo” (schedulingzoo.lip6.fr) 把三段式记号做成了可查询的复杂度表 , 可以随时对照本章的图 9-3。

本章 Lab

代码包 `ch09-scheduling-language-lab.zip` 随本章一起发放。依赖是 `numpy`、`pytest` ; 判分用暴力枚举 , 不依赖你的代码。

练习	内容	对应
ex1	目标函数 : 评估一个顺序的 $C_{\max}$ 、 $\sum C_j$ 、 $\sum w_j C_j$ 、 $L_{\max}$ 、 $\sum T_j$ 、 $\sum U_j$ (含释放时间) ; 枚举每个目标的最优 ; 先填 4 个预测	9.1、9.3
ex2	并行机 : 列表调度 (P、Q、R 通用的 p_{ij} 表) 、枚举最优 $C_{\max}$ 、下界	9.4
ex3	记号 : 解析 $\alpha \mid \beta \mid \gamma$, 判断“是不是另一个问题的特例”	9.2、9.5
ex4	复杂度推理 : 由已知结果表沿特例与推广推出分类	9.5
ex5	审 5 个 AI 写的目标计算函数 : 不看源码 , 只靠运行结果归类问题	9.3
挑战	$\sum C_j$ 与 $L_{\max}$ 的帕累托前沿	9.3

```

pip install -r requirements.txt
pytest -q                                # 全部练习
pytest tests/test_ex3.py -q             # 单个练习

# 确认判分本身没问题 : 参考答案应当全部通过
LAB_TARGET=solutions pytest -q

```

练习 1 要求先在 `PREDICTION` 里写下预测再运行 ; 跑完测试后 , 终端最后会把你的预测和实际结果并排列出来。

下一章

CHAPTER 10

单机与并行机

本章反复出现的 SPT、WSPT、EDD、Moore-Hodgson，都是“一条规则就得到最优”的例子。下一章逐个证明它们为什么对：用交换论证，比较相邻的两个作业，一次交换不会变坏。再看并行机上的列表调度，它的最坏情况有多差，近似比是多少。

9.8 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 “ $1 | r_j | L_{\max}$ ”这个标签的意思是？

一台机器、有 r 个作业、最小化最大长度

单机、可中断、最小化最大完工时间

并行机、有释放时间、最小化延迟之和

单机、作业有释放时间、最小化最大延迟

正确。 $\alpha = 1$ 是单机， $\beta = r_j$ 表示每个作业有释放时间（不能在它到达前开始）， $\gamma = L_{\max}$ 是最大延迟 $\max(C_j - d_j)$ 。

Q2 $L_{\max}$ 和 $\sum T_j$ （延误时间之和）的区别是？

没有区别

$L_{\max}$ 看最坏的一单，可以是负数（提前）； $\sum T_j$ 把每一单的迟到时间相加，提前不算（不奖励）

$L_{\max}$ 只用于并行机

$\sum T_j$ 不受顺序影响

正确。 $L_j = C_j - d_j$ 可以是负数，取最大值； $T_j = \max(0, L_j)$ ，求和。所以 $L_{\max}$ 最优的排法（EDD）不一定让 $\sum T_j$ 最小。

Q3 图 9-1 里，为什么“SPT”“WSPT”“EDD”各自最优的顺序不一样？

因为交货期是随机的

因为算法有 bug

它们分别是 ΣC 、 ΣwC 、 $L_{\max}$ 的最优规则；目标不同，偏好的顺序就不同，没有一个顺序同时最好

因为加工时间是随机的

正确。 SPT 让短作业早完成，降低完工时间之和；WSPT 兼顾权重；EDD 让交货期近的先做，压低最大延迟。它们互相冲突， ΣC 与 $L_{\max}$ 的帕累托前沿有 8 个点。

Q4 没有释放时间的单机上，为什么 $C_{\max}$ 与作业的顺序无关？

因为 $C_{\max}$ 不是目标

因为顺序总是最优的

因为作业可以同时做

机器一直不空闲，最后一个作业的完工时间总是总加工时间

正确。 没有释放时间和准备时间，机器从 0 开始连续加工，全部完成的时刻等于加工时间之和。有释放时间或与顺序有关的准备时间时，就不再无关了。

Q5 P、Q、R 三种并行机的关系是？

P 是 Q 的特例（速度都为 1），Q 是 R 的特例（ $p_{ij} = p_j / s_i$ ），所以 R 最一般

R 是 P 的特例

Q 是 P 的特例

三者互不相关

正确。 相同并行机的加工时间与机器无关；速度不同的机器加工时间成比例；不相关机器是任意的表。越一般的环境，问题越难（或一样难）。

Q6 已知 $P2 \parallel C_{\max}$ 是 NP 难的。据此可以推出？

1 $\parallel C_{\max}$ 是 NP 难

$R \parallel C_{\max}$ ($P2$ 的推广) 也是 NP 难

$R \parallel C_{\max}$ 是多项式的

什么也推不出

正确。 难度沿着推广传播：解 $R \parallel C_{\max}$ 的算法可以直接用来解 $P2 \parallel C_{\max}$ 。 $1 \parallel C_{\max}$ 是 $P2$ 的特例，没有这个结论（它其实是平凡的）。

Q7 $R \parallel \Sigma C_j$ 是多项式可解的，而 $R \parallel C_{\max}$ 是强 NP 难。这说明什么？

ΣC_j 是 $C_{\max}$ 的推广

机器环境决定了问题的难度

同一个机器环境里，目标不同，难度可以完全不同

R 环境不可解

正确。 $R \parallel \Sigma C_j$ 可以化成指派问题（第 5 章），是多项式的。 $R \parallel C_{\max}$ 是均衡各机器负荷，本质上是装箱类的问题，强 NP 难。

Q8 “ $1 \mid s_{jk} \mid C_{\max}$ ”（单机、与顺序有关的准备时间、最小化完工时间）对应的经典问题是？

指派问题

最短路

背包问题

旅行商问题：作业是城市， s_{jk} 是从 j 到 k 的距离

正确。 准备时间只取决于前后两个作业，总完工时间 = 加工时间之和 + 准备时间之和，问题就是找一条最短的访问顺序，即旅行商（第 8 章）。设备在作业之间要移动时就是这种模型。

8 / 8 本章掌握得不错，可以去做 Lab 了。

数据说明：本章的全部数字由页面内置的目标函数评估、720 种顺序的枚举、并行机列表调度与枚举、复杂度地图的推理实时计算，并与 Lab 的判分数据同源；复杂度结果表来自教材中的经典结果（Pinedo、Brucker 等）（环境：SciPy 1.17.1 · HiGHS 1.12.0）。